BookBridge Collection Strategy

LibraryService exclusively owns the mutable collections. Book, loan, and reservation records and the three supplied member subclasses are immutable. There are no raw collections or unchecked casts.

books: Map<String, LibraryBook>

Implementation: LinkedHashMap

Average O(1) ISBN lookup, deterministic insertion order; duplicate normalized ISBN rejected; replace status snapshot on issue/return

members: Map<String, Member>

Implementation: LinkedHashMap

Average O(1) member ID lookup with deterministic registration order; duplicate normalized ID rejected

activeLoans: Map<String, LoanRecord>

Implementation: LinkedHashMap

Average O(1) loan ID lookup, issuance order; insert on issue, remove on return

pendingReservations: Queue<Reservation>

Implementation: ArrayDeque

Efficient append, insertion-order iteration, no nulls; remove only fulfilled/cancelled entry, preserving remaining relative order

checkedOutISBNs: Set<String>

Implementation: HashSet

Average O(1) membership/uniqueness; one ISBN per active loan; add on issue, remove on return

loanHistory: List<LoanRecord>

Implementation: ArrayList

Issuance-order reporting; append issued records, replace returned record at its original position; includes both active and completed loans

Auxiliary usedIds: Set<String> uses HashSet to reject loan/reservation ID reuse, even after return or cancellation. L-/R- prefixes distinguish namespaces. UUIDs avoid a process-wide static counter. A Supplier<String> can later provide persisted IDs. Cancellation does not make an ID reusable.

EnumMap accumulates all membership groups, including zero counts; reports return an immutable copy. Reservation processing needs no extra tracking collection: scanning in queue order and returning after the first successful issuance preserves FIFO for each book.

State invariants

1. Active-loan ISBNs are unique and exactly equal to checkedOutISBNs.

2. A registered book is CHECKED_OUT exactly when its ISBN has an active loan.

3. Every active loan has exactly one matching ACTIVE history record. Returned records remain in history and are absent from activeLoans.

4. Every loan/reservation references registered members/books. Pending member/ISBN pairs and generated IDs are unique.

5. All new books are AVAILABLE; clients cannot import inconsistent checked-out state.

The consistent assertion helper checks these relationships after every service scenario, including rejected operations.

Mutation strategy

Every operation validates entities, input, duration, availability/queue fairness, and conflicts before changing collections. Values, dates, and IDs are prepared before the commit block. A failure in ID generation or a backward return clock propagates before updates. No business validation or injected callbacks occur inside commit blocks.

- Issue: validate duration in issueLoan (or processNextReservation), then prepare loan and CHECKED_OUT book; update active map, history, ISBN set, book map, and used IDs.

- Return: find active loan and its history index, validate current status, prepare returned record and AVAILABLE book; replace history, remove active/set entries, replace book. Old loan IDs cannot release subsequently reissued books.

- Reserve: validate registered entities and duration, reject duplicates/self-held/available unqueued books, prepare ID/value; append and retain ID.

- Cancel: validate ID, find via Iterator, remove exactly that entry. Missing IDs throw without mutation.

- Process: scan using Iterator, leave unavailable entries in place, prepare/commit the first eligible loan, then iterator.remove() its reservation and return. In the supported single-threaded use, removal cannot conflict with another iterator. If nothing is fulfillable, return Optional.empty(). A failed preparation preserves the queue.

This strategy protects expected validation and creation failures in a single-threaded in-memory program. It does not claim database transactions, thread safety, or recovery after JVM failure. Future concurrency requires locking or transactional persistence around whole operations.

Fairness

Example queue: F waits for A, G waits for A, G waits for B. If A is still checked out but B has been returned, G receives B; both A entries retain order. When A returns, F receives it first. Direct issueLoan cannot bypass pending entries for that ISBN, including by the first waiting member. That member uses processNextReservation instead. Return does not automatically process the queue.

Encapsulation, generics, and traversal

List.copyOf, Set.copyOf, Map.copyOf, and Stream.toList return immutable snapshots. The supplied model values are immutable too, preventing mutation through a collection element. Set/map report iteration order is not promised; explicit list ordering is promised.

Lookup.require(Map<K,V>, K, String) is the useful generic abstraction shared by member/book/active-loan lookups. It preserves key/value types and produces clear missing-entity errors without casts or three repeated implementations.

An Iterator supports cancellation and fulfillment because it removes the selected entry without reordering or fail-fast modification during traversal. Enhanced for loops initialize/accumulate membership counts. Streams provide readable title/status filters and fine totals. A chained Comparator sorts a new list by active overdue first, due date, then ID; source history is unchanged. Records express immutable domain/report values and method references avoid repetitive lambdas. Member remains an ordinary abstract class like TSA2; its supplied final subclasses have fixed policies and no mutable fields.

Registry and ISBN-set lookups are average O(1). Issuance scans pending reservations in O(queue size); creating the loan does not repeat the duration check or scan active loans. Reservation creation scans active loans and the queue to reject self-held and duplicate requests. Return finds history position in O(history size); cancellation/processing are O(queue size). Sorting is O(history size log history size). These costs are suitable for this in-memory assessment; indexes can be introduced later if profiling justifies them.

AI assistance disclosure

OpenAI Codex assisted with implementation, review, testing, and documentation. The student should review and understand the submitted work. See AI_ASSISTANCE_DISCLOSURE.md for the full disclosure.